Decision Guidance Models for Microservice Monitoring

Stefan Haselböck and Rainer Weinreich
Johannes Kepler University Linz
Linz, Austria
{stefan.haselboeck, rainer.weinreich}@jku.at

***Abstract*— Decision guidance models are a means for design space exploration and documentation. In this paper, we present decision guidance models for microservice monitoring. The selection of a monitoring system is an essential part of each microservice architecture due to the high level of dynamic structure and behavior of such a system. We present decision guidance models for generation of monitoring data, data management, processing monitoring data, and for disseminating and presenting monitoring information to stakeholders. The presented models have been derived from literature, our previous work on monitoring for distributed systems and microservice-based systems, and by analyzing existing monitoring systems. The developed models have been used for discussing monitoring requirements for a microservice-based system with a company in the process automation domain. They are part of a larger effort for developing decision guidance models for microservice architecture in general.**

Keywords— decision guidance models; microservices; microservice monitoring

I. Introduction

Microservices is an architectural style for building software systems from a set of services, which can be independently developed, deployed and operated [1], [2]. Introducing a microservice infrastructure leads to various challenges in decision making regarding microservice system design, team organization, development processes, and infrastructure [3], [4]. One major requirement when introducing a microservice architecture is the establishment of a microservice monitoring infrastructure [1], [3].

Decision guidance models are a well-known means for design space exploration and decision guidance in software architecture. Decision guidance models have been presented for design space exploration in domains like cyber-foraging systems [5], and cloud computing [6]. We are currently working on decision guidance models for microservices. In this paper, we present a part of this work, which are decision guidance models for establishing a microservice monitoring infrastructure.

The created decision guidance models are intended to be used in several ways. They can be used for

1. identifying important requirements for a microservice infrastructure,
2. selecting the best design options for addressing a set of requirements,
3. comparing design options based on consequences resulting from their selection (implications),
4. identifying concerns (requirements, aims, stakeholder needs) that are addressed by selecting a particular design option,
5. identifying important system components (e.g., an aggregation service), which are required to implement a specific design option,
6. identifying the design options that are implemented by a particular monitoring technology, and for
7. comparing existing monitoring technologies with respect to the support of a specific design option and/or requirement.

The presented decision guidance models were created based on a literature review, on our previous work on monitoring, and by comparing existing monitoring technologies. The created models were tested in a workshop with a company in the automation domain, which is currently introducing a microservice architecture.

The remainder of the paper is organized as follows: Section II presents the research approach. In Section III we present an overview of important concepts related to monitoring in general and to microservice monitoring in particular. In Section IV we present the notation we use for representing the decision guidance models that are presented in the following sections. In Sections V-VIII we describe the developed decision guidance models. We describe related work in Section IX. Section X contains a conclusion and an outlook on further work in this area.

II. Research Approach

To create the decision guidance models presented in this work, we performed the three consecutive steps: creation of initial models, model validation, and model refinement.

A. Creation of Initial Models

To create the initial decision guidance models, we first performed a literature review on microservices and on decision guidance models. Based on the knowledge gained from the literature review, our previous work on monitoring of distributed object-oriented systems [7], our current work on monitoring of microservice-based systems [8], and on analyzing related monitoring systems like *Dynatrace APM [9], New Relic [10], Zipkin [11],* and the *Elastic Stack [12]* we created decision

978-1-5090-4793-2/17 $31.00 © 2017 IEEE
DOI 10.1109/ICSAW.2017.31

IEEE computer society

Authorized licensed use limited to: University of Applied Sciences Karlsruhe c/o KIT Library. Downloaded on October 28,2026 at 10:55:38 UTC from IEEE Xplore. Restrictions apply.

guidance models for different areas of microservice monitoring like data collection, processing, dissemination and presentation.

B. Model Validation

To test the created decision guidance models, we participated in a workshop for microservice design at one of our partner companies (a company in the process automation domain). Meetings like this take place every second week and always treat a different challenge for establishing a microservice architecture. In the meeting we took part, the aim of the client was to get an overview of potential requirements for microservice monitoring and of existing monitoring technologies.

During the workshop the models were used for discussion. We acted as observers and data collectors, and only answered concrete questions about monitoring. Furthermore, we took field notes [13] of our observations.

To analyze the created field notes we coded them using a provisional start list [14] with one category for each different element (concern, design option, implication, component, or technology option) of the model. On the basis of the coded field notes we created a field memo [13] with the required changes to the decision guidance models.

C. Model Refinement

We refined the decision guidance models for microservice monitoring on the basis of the created field memo. This included changes to the model like adding new requirements or aims, changing existing ones, adding and changing implications of a design option, or adding and changing relations between different decision guidance models.

III. Monitoring

In general, monitoring is the observation of an executed system [15]. A separate process, the monitor, is responsible for observing the system based on data collected by various sensors (or agents) [15]. Regarding service-based applications Benbernou et al. [16] describes monitoring as the process of collecting and reporting relevant information about the execution and evolution of an application. The monitoring process can be used to achieve different goals. Some examples are: runtime optimization of a system, detecting relevant runtime changes, and collecting information for system evolution [16].

A. Monitoring Activities

According to Mansouri-Samani et al. [17] operating a monitoring system consists of four consecutive activities:

1) Generation of Monitoring Information

The first step is the generation of required monitoring information through object instrumentation, i.e., through placing probes, or sensors into the objects of interest, which then record certain activities or state changes in a system, typically in the form of events. The generated information then can be used for different kinds of analysis [17].

2) Processing Monitoring Information

Processing monitoring information starts with the validation of the generated information. Afterwards, monitoring information is combined, filtered, and analyzed to detect the cause for a problem or to determine the current system state [17].

3) Dissemination

The generated monitoring reports are then delivered to interested users. Dissemination can follow a fixed scheme, e.g., by forwarding the information to all system users, or a variable scheme, meaning that the information is only forwarded to users who subscribed for all or only a subset of the generated events [17].

4) Presentation

The generated and processed monitoring information has to be presented in a way to meet the stakeholder requirements. Therefore, a suitable user interface has to allow the stakeholders to specify the way how to display the information [17].

B. Monitoring of Microservices

Monitoring microservice-based systems is an especially challenging task due to the fact that the services are running in independent processes spread over different hosts [1]. This requires a monitoring system that automatically collects, distributes, and processes monitoring information in a system with a high number of distributed components [1].



Fig. 1. Potential stakeholders and concerns for a microservice monitoring system

In Fig 1. we present potential concerns and stakeholders for a microservice monitoring system. The list is split into potential aims, potential requirements, and potential stakeholders. Aims

Authorized licensed use limited to: University of Applied Sciences Karlsruhe c/o KIT Library. Downloaded on October 28,2026 at 10:55:38 UTC from IEEE Xplore. Restrictions apply.

refer to the need of potential business functionality, requirements are regarding the required monitoring information for specific kinds of analysis, and stakeholders are potential users of the system. The requirements, aims, and stakeholders presented in Fig. 1 are the result of our literature review and of comparing existing systems for microservice monitoring and has been refined in one workshop with practitioners. We are aware that this list is by no means complete, but it is a first step to identify basic requirements for such a monitoring system.

C. Microservice Monitoring Areas

On the basis of the aforementioned monitoring activities we defined four major areas for microservice monitoring.

1) Generation of Monitoring Data

The first area is the same as described by Mansouri-Samani et al. [17]. The area includes the generation of monitoring data by instrumenting hosts, runtime environments, and services.

2) Data Management

In addition to the activities of Mansouri-Samani et al. [17] we identified data management as an important area of a monitoring system. This area contains the storing, hosting, and distribution of monitoring data. These steps are necessary to provide the generated data to the following processing activities.

3) Processing of Monitoring Data

This area includes the processing of the generated monitoring data to gain information about service interactions, service runtime metrics, and infrastructure runtime metrics. It also includes activities like aggregating monitoring data over time to deal with the huge amount of generated monitoring information, and to enable long-term reports.

4) Dissemination and Presentation

Since dissemination and presentation are closely related in a microservice monitoring system, we combined them into one area. It includes the offering of a user interface for presenting important monitoring information to stakeholder.

IV. Decision Guidance Model Elements and Relations

Fig. 2 presents the notation we used for defining microservice decision guidance models described in the following sections.

As shown in the figure, the main elements are concerns (yellow box), design areas (gray box), design options (blue box), implications (green box), components (orange box), and technology options (red box).

Each microservice-monitoring area of design is represented by one or more decision guidance models. As described in the section before, concerns can be potential aims, requirements, and other stakeholder needs that may drive the exploration of a design area. Design options are potential solution to address concerns that may have implications (positive or negative) on system qualities, organizational structures, and infrastructure. In the case of the decision guidance models presented in this paper, implications mostly relate to impacts on system qualities or system administration. Design options can be alternative to or may complement another design option. Furthermore, they can require specific system components for their implementation. Technology options are specific technologies realizing a specific design option or providing a specific component.

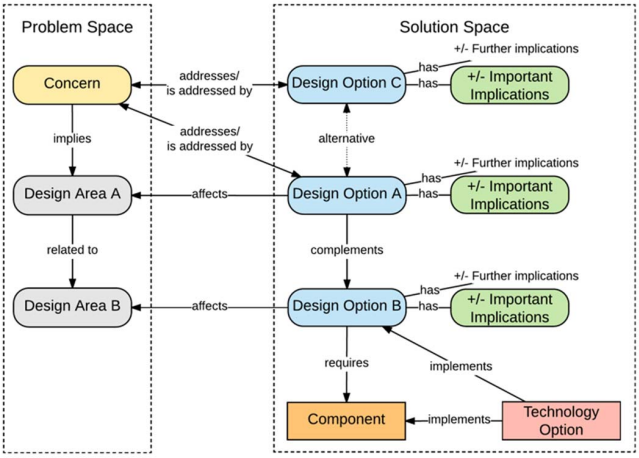


Fig. 2. Decision Guidance Model Notation: Elements and Relations

V. Generation of Monitoring Data

Fig. 3 shows the decision guidance model for generation of monitoring data, which mainly contains design options for instrumenting services to collect monitoring data.

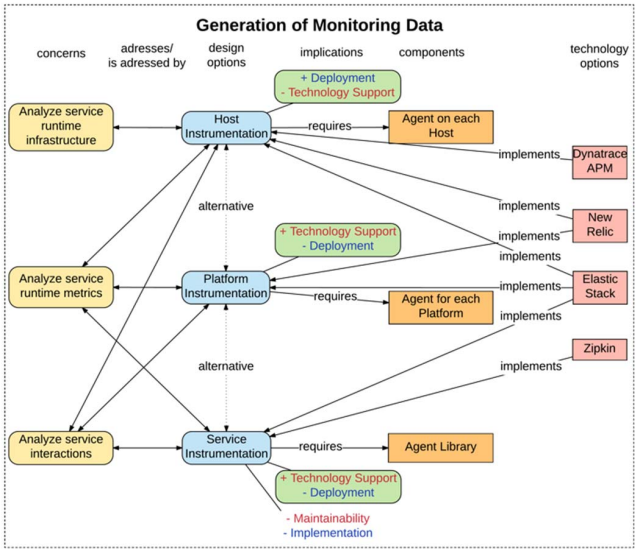


Fig. 3. Decision Guidance Model for Generation of Monitoring Data

The three main concerns in the model are functional requirements and refer to different kinds of monitoring data that need to be collected. The first requirement refers to the need for collecting information on the *service runtime infrastructure* like available hosts and datacenters, CPU and memory consumption per host, and how service instances map to the underlying infrastructure. The second requirement refers to *service-specific metrics* like response time, failure rate, and throughput, as well as CPU and memory consumption per service. The third requirement refers to the need for analyzing *service interactions* like communication paths between services, or runtime coupling

Authorized licensed use limited to: University of Applied Sciences Karlsruhe c/o KIT Library. Downloaded on October 28,2026 at 10:55:38 UTC from IEEE Xplore. Restrictions apply.

of services. Analyzing service interaction can be facilitated for root cause analysis.

The system to be monitored can be instrumented at three different levels to generate the required data: at the *host level*, the *platform level* and at the *service level*. These three options are represented as exclusive design options in the model.

Host Instrumentation requires an agent for collecting runtime-related data to be installed at each host of a microservice runtime infrastructure. This agent automatically detects new microservice instances and starts collecting monitoring data. Using this design option each host has to be instrumented, e.g. by instrumenting low-level communication libraries. Since only the host is instrumented, it is not possible to generate specific data for particular runtime environments or services. However, this design option is beneficial from the perspective of agent deployment, since agents need only be deployed per host and service developers do not need to deal with instrumentation.

Using *Platform Instrumentation* means that each service runtime environment, e.g., each Tomcat or MySQL server, has to be instrumented. This requires an *agent for each platform* used at a specific host. The agent implementation is independent from the operating system and it can be more technology-specific than an agent at the host level. An agent for the *Java Virtual Machine (JVM)*, for example, can collect data on garbage collection and memory usage within the JVM. However, the need of one agent for each platform requires the deployment and operation of several agents on each host.

Using *Service Instrumentation* requires an agent for each service. This allows the analysis of service runtime metrics, and service interactions, but provides no information about the service runtime environment. The implementation of this design option typically requires an *agent library* for each service, which needs to be deployed with each service. In addition, instrumentation has to be taken into account during service implementation, which means an additional burden for the service developer.

Fig. 3 also contains elements representing technology options that are available for implementing the different kinds of instrumentation (red boxes). *Dynatrace APM* uses host instrumentation to generate the required monitoring data and thus requires an agent per host (orange box). *New Relic* implements platform instrumentation to analyze service runtime metrics and service interactions, and it implements host instrumentation to generate data for service runtime infrastructure analysis. Thus, it requires one agent per platform, and also one agent per host. Depending on the configuration the *Elastic Stack* provides platform or service instrumentation, and in addition, it implements host instrumentation to generate data for service runtime infrastructure analysis. Thus, it requires either one agent per platform or an agent library, and one agent per host. *Zipkin* implements service instrumentation and thus requires an agent library for each service.

VI. Data Management

After the generation of monitoring data, the next step is to manage the generated data. For this activity, we have captured design options in three different models shown in Fig. 4. We distinguish between design options for determining the storage location (centralized/decentralized), for selecting a hosting strategy (self-hosted or SaaS), and for selecting a distribution strategy (push or pull).

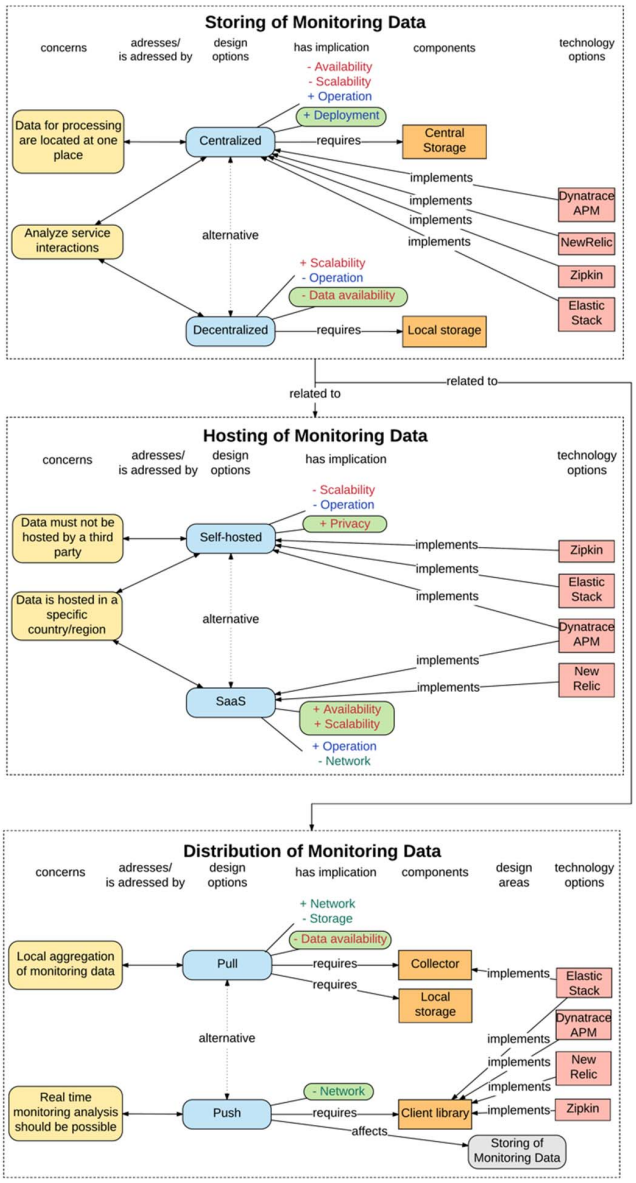


Fig. 4. Decision Guidance Models for Storing, Hosting, and Distribution of Monitoring Data

A. Storing of Monitoring Data

There are two main concerns that may drive the selection of a *centralized storage* vs. a *decentralized storage* of monitoring data. The first is the requirement for *storing data at one place* and the second is the requirement for *supporting analysis of service interactions and relations*. Storing data in one place may be required, because collecting data from several distributed places (e.g., nodes in a system) before processing requires additional time for data collection. Supporting analysis of service interactions may be required for root cause analysis.

Authorized licensed use limited to: University of Applied Sciences Karlsruhe c/o KIT Library. Downloaded on October 28,2026 at 10:55:38 UTC from IEEE Xplore. Restrictions apply.

The decision guidance model for storing of monitoring data offers two alternative design options. Both design options are exclusive, which means that only one design option out of the two can be chosen.

With the *Centralized* design option, monitoring data of all running microservices are stored at one place. This requires a *central storage* component. A central storage component allows the analysis of service interactions in one place, since the data required is centralized. It also means less administration overhead, since the deployment and operation of the storage has to be performed only once. Also, the monitoring data of new services can be stored into the already existing storage. However, especially in larger system the storage might become a bottleneck and thus needs to support up-scaling. A second tradeoff is that the storage is also a single point of failure. If it is not available, monitoring data cannot be stored and thus analysis is not possible.

The *Decentralized* design option is based on a local storage component at each host, platform or service. Analyzing service interactions in this case is only possible, if the monitoring data of different local storages are collected before data processing. A disadvantage of this design option is that the unavailability of a local storage (e.g., due to a crashed host) will result in a loss of monitoring data. Furthermore, operating many local storages needs a lot of administration effort. This design option is beneficial from the perspective of scalability, since installing new services at new hosts will also provide a new local storage and scaling up the storage as in the centralized solution is not necessary.

The technology options shown in the model all implement the *Centralized* design option.

B. Hosting of Monitoring Data

The main requirements of the decision guidance model for hosting of monitoring data in Fig. 4 are (1) keeping monitoring data within the company and thus avoiding a third-party solution and (2) restricting the storage of monitoring data to a specific country or region.

The decision guidance model provides two alternative design options: self-hosted and using a service by a third-party provider (SaaS). Using the *Self-hosted* option the storage for monitoring data is hosted by the company itself. Thus, this design option meets the requirements that data must not be hosted by a third party, and potentially (since this can be decided by the company) that data is hosted in a specific country. Though self-hosting has a high privacy level, extra effort is necessary to operate the required storage. Furthermore, up-scaling of the storage can be a time consuming and cost extensive task.

With the *Software-as-a-Service* (SaaS) solution, the company uses the infrastructure of a SaaS provider. This enables on-demand scalability, calculable costs based on a defined accounting model, and high availability as often defined in a service level agreement. Consequently, the company does not have to deal with operating the infrastructure. On the other hand, using a SaaS solution may potentially result in higher network traffic, as all monitoring data has to be sent to the provider. This solution also requires a level of trust between company and service provider and the company has to take care that no sensitive data like user names or passwords are transferred to the service provider as part of the monitoring data.

The technology options shown in the model use different hosting strategies. *Zipkin* and the *Elastic Stack* only provide a solution that has to be self-hosted. In contrast, *New Relic* provides their solution only as a SaaS system. *Dynatrace APM* offers both options.

C. Distribution of Monitoring Data

The distribution decision guidance model contains the potential concerns *local aggregation of monitoring data* and *real time monitoring analysis should be possible*. The first is a requirement and refers to the need for reducing network traffic by aggregating monitoring data locally on a host. The second is also a requirement and refers to the need for analyzing monitoring data such as service runtime metrics, service interactions, or infrastructure-related data in real time.

To address the aforementioned concerns the model provides two alternative design options, *Pull* and *Push*. Pull is designed to collect data from a local storage (e.g., a log file) and either store it in a central storage, or make it available for further processing and presentation. Since data is first stored in a local storage, this option offers the opportunity to aggregate data locally to reduce network traffic. If, however, the local data is destroyed due to a crash of a service or container before it has been pulled to the central storage, this data may be lost.

In contrast to pull, using *Push* all data is sent immediately to a central storage by the agents generating monitoring data. Thus, choosing this design option affects the storage of monitoring data in the sense that only the design option *Centralized* can be chosen. Sending all monitoring data directly to the central storage causes a high number of network calls, and does not allow any local aggregation. On the other hand, the design option enables real time monitoring of service-, or infrastructure-related data.

All technologies shown in the model support *Push* as a design option. Only *Elastic Stack* also provides support for *Pull* as a design option using a collector component.

VII. Processing of Monitoring Data

Fig. 5 shows the decision guidance model for processing of monitoring data. The model contains design options regarding capturing service interactions, the level of aggregation of stored monitoring data, and threshold analysis.

The set of concerns shown in the model in Fig. 5 contains a number of high-level aims and requirements in terms of functionality to be provided and required monitoring information.

Root cause analysis refers to the need for identifying the root cause in case of failure of a service. *Analyze service interactions* refers to the need for information for analyzing communication and interaction between services. This is typically a prerequisite for root cause analysis. *Analyze service runtime metrics* refers to the need for information about service response time, failure rate, throughout, and CPU and memory consumption. A*nalyze service runtime infrastructure* refers to monitoring information about available hosts and datacenters in the service

Authorized licensed use limited to: University of Applied Sciences Karlsruhe c/o KIT Library. Downloaded on October 28,2026 at 10:55:38 UTC from IEEE Xplore. Restrictions apply.

infrastructure, about how services are allocated to infrastructure nodes, and about CPU and memory usage per host. G*eneration of long term reports* refers to the need for providing and analyzing monitoring information over a long period of time (e.g., to identify long-term trends). Finally, the last two requirements refer to the need for alerting functionality to notify users in case of exceeded thresholds or deviation from normal operation.

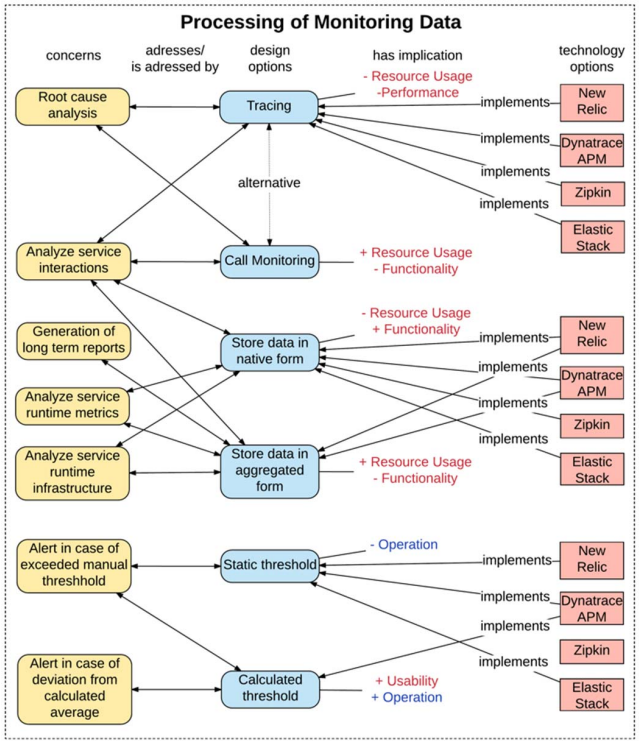


Fig. 5. Decision Guidance Model for Processing of Monitoring Data

The model for processing monitoring data contains six different design options, which can be classified into the three groups service interaction analysis, aggregation of monitoring data, and threshold alerts.

Design options for providing information on service interaction include tracing and call monitoring. Tracing allows to determine the initiator of a sequence of service calls and is thus a means for root cause analysis. Different kinds of tracing exist [18], which differ mainly in the level of intrusiveness of an instrumented system. For example, black-box approaches like [19] can be used in systems where only minimal or no modification of monitored components is possible, while annotation-based monitoring requires some form of tagging communication (e.g., with time stamps or correlation IDs). The required overhead for tagging and building communication traces depends in the implementation of the respective tracing system. This means resource utilization and performance overhead of such systems are also important. *Call monitoring* is a simple method for just logging information on which service (or service instance) called another service (instance). This is similar to black-box tracing, however no trace information is generated. Instead, only the number of incoming and outgoing calls per service is stored. If this is combined with information about a service API this can be used for determining call frequency on the level of service instances and on the level of service types (if aggregated). It also allows analysis of communication frequency between different services.

The second category offers the design options *store data in native form*, or *in aggregated form*. Storing data in native form enables detailed analysis. On the other hand, this may lead to a huge amount of collected monitoring data, which may need a huge amount of storage, making it almost impossible to store data over a long period of time. Thus, this design option is mainly useful for short time failure analysis, but it does not support the generation of long term reports. Storing data in *aggregated form* is a way for reducing the required amount of storage. Because of this, it allows long term analysis, since data for long term reports does not have to be as detailed as data for root cause analysis. However, depending on the level of aggregation, some kinds of analyses are not possible with this design option. For example, root cause analysis requires detailed information on service communication and this is not possible with aggregated request data. Since the two design options are not mutually exclusive they can also be used in combination. Data in native form can be used for short term analysis, while aggregated data can be used for long term analysis.

For alerting-functionality, the decision guidance model provides the design options *static threshold*, and *calculated threshold*. Both design options enable the notification of responsible persons in case of exceeded thresholds. Using static thresholds alerts are sent if the manually preconfigured threshold is exceeded. Using a calculated threshold, the monitoring system learns based on previous monitored data and automatically reconfigures the threshold. This enables alerts if the actual value deviates from a value during normal operation.

For the sake of presentation, we duplicated technologies for each of the above described categories in the model shown in Fig. 5. All technologies in Fig. 5 support tracing and thus focus on short-time analysis. *New Relic* and *Dynatrace APM* also support long-term analysis through aggregation. Threshold alerts based on manual thresholds are supported by *New Relic*, *Dynatrace APM*, and *Elastic Stack*. *Dynatrace APM* also supports calculated thresholds.

VIII. Dissemination and Presentation of Monitoring Data

After processing, the monitoring data needs to be disseminated and presented to the stakeholders. The decision guidance models shown in Fig. 6 offers design options for dissemination and presentation.

A. Dissemination of Monitoring Data

In terms of dissemination of the provided monitoring information to a particular user, we distinguish between (1) how the user can access the data (Web, Mobile, Web), (2) whether the information can be tailored to a particular stakeholder, and (3) whether information from different backend systems can be seamlessly integrated.

Authorized licensed use limited to: University of Applied Sciences Karlsruhe c/o KIT Library. Downloaded on October 28,2026 at 10:55:38 UTC from IEEE Xplore. Restrictions apply.

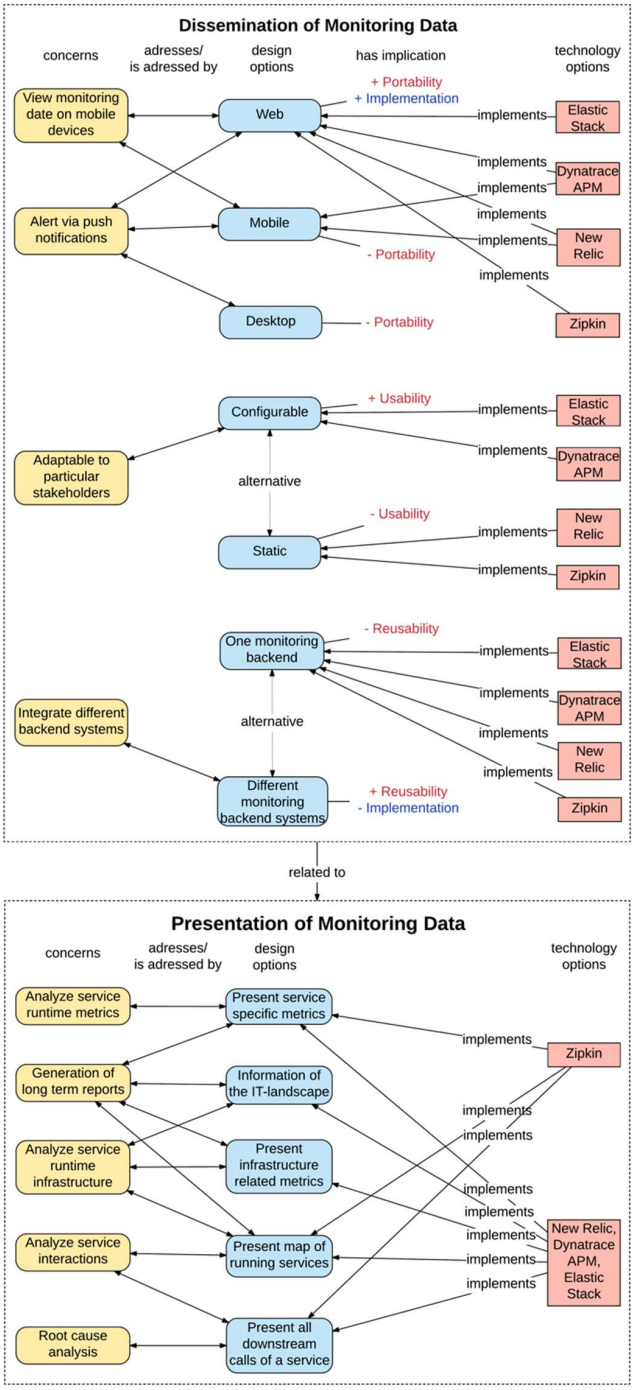


Fig. 6. Decision Guidance Models for Dissemination and Presentation of Monitoring Data

The data can be presented to a user either through a Web-based user interface, through a desktop application or through an application on a mobile device. All of these options can be realized by a particular system. We also identified two potential concerns that were important to stakeholders in this regard: the availability on mobile devices and the support for push notifications in case of alerts. Presentation on mobile devices can be supported by a dedicated application for a mobile device or by a web application supporting mobile web design or responsive web design. Alerting can be supported for all kinds of applications.

If adaptability to particular stakeholders and their concerns is required, a monitoring solution needs to be configurable in terms of the UI and data presented to end users. A configurable solution allows a particular stakeholder to adapt the presented information to his or her needs. A static solution only provides preconfigured views for particular kinds of stakeholders and cannot be tailored to a particular stakeholder's needs if required.

For the integration of backend systems, the model defines a requirement to integrate different backend systems. This may be required, if different monitoring infrastructures are operated in parallel and monitoring results should be presented in one central place to a stakeholder.

To integrate monitoring data the decision guidance model in Fig. 6 offers two design options. The first option only allows to integrate one monitoring backend, and it is not possible to analyze and compare data of different systems. Alternatively, the other design option allows using different monitoring backend systems. Using this design option requires an extra effort for integrating different backend systems.

As in the model for processing monitoring data, we duplicated the elements representing monitoring technologies for each of the above described categories in the model shown in Fig. 6. All technology options in the model implement the web design option regarding user access. Additionally, *Dynatrace APM* and *New Relic* also implement user access through mobile devices. A configurable dashboard to tailor information to a particular stakeholder is only offered by Elastic Stack and *Dynatrace APM*. The integration of data is not offered by any of the technology options. They only support their own backend system. Some monitoring technologies support importing monitoring data from other sources.

B. *Presentation of Monitoring Data*

The five potential concerns in the model for presenting monitoring data largely resemble the concerns that are part of the model for processing monitoring data described in the previous section. The design options contained in the model now contain elements for presenting information to address these concerns instead of design options that are needed for processing monitoring data to be able to deliver the required monitoring information.

The five design options in the model can be used in combination. The design option *present service specific metrics* shows service runtime metrics for analyzing response time, failure rate, throughput, as well as CPU and memory consumption per service. If such a component is present it may support long-time metrics, if this is supported by the processing layer. *Presentation of the IT-landscape* allows analysis and generation of long term reports of infrastructure-related monitoring data like available hosts and datacenters, host utilization, or service allocation to hosts. *Present infrastructure related metrics* allows to analyze metrics like CPU load and memory consumption for a specific host. *Present map of running services* provides an overview of all running services,

Authorized licensed use limited to: University of Applied Sciences Karlsruhe c/o KIT Library. Downloaded on October 28,2026 at 10:55:38 UTC from IEEE Xplore. Restrictions apply.

of service interactions, and potentially provides long term reports for these data. Using *present all downstream calls of a service* supports finding the root cause in case of service failure, if tracing is supported by the generation and processing layer. Otherwise, it supports analysis of service interactions based on call frequencies.

With the exception of *Zipkin*, all examined monitoring technologies support the aforementioned design options. *Zipkin* provides no information on the service runtime infrastructure since this is not supported at the generation and processing layer in *Zipkin*.

IX. Related Work

Question-Option-Criteria (QOC) [20], [21] defines basic concepts for design space exploration and decision models. Using QOC, questions are key problems structuring the design space, options are possible alternative solutions, and criteria are used for evaluating and choosing options. On the basis of QOC, Zdun [22] proposes decision models for systematic pattern selection in design space analysis by creating design spaces based on questions referring to design problems, patterns and their variants relating to options, and criteria relating to forces (i.e., system quality goals).

As far as we know, there is no related work on design space exploration for microservice monitoring as presented in this paper. Nevertheless, decision guidance models have already been used for design space exploration in other domains like service-oriented systems [6], cyber-foraging systems [5], and cloud computing [6]. We used some elements of these models as a basis for identifying the elements and relations in our decision guidance models. To meet the special requirements regarding decision making when introducing a microservice architecture, we refined and extended these previous decision models through a technical action research process with our partner companies. As part of this process, we also used the decision guidance models presented in this paper within one client engineering cycle with a company in the process automation domain.

X. Conclusion

We presented a series of decision guidance models that can be used when introducing a microservice monitoring system. The models have been created based on literature, our experience in this area, and on existing commercial and open source tools for microservice monitoring. They have been tested with a company, which is currently exploring the introduction of a monitoring system during the establishment of a microservice architecture. The models were especially considered to be useful for exploring requirements for such a monitoring infrastructure and for comparing existing technology options as discussed in the paper.

In general, the development of the described decision guidance models is part of a larger effort for creating decision guidance models for various aspects of microservice architecture at our department. We also view the decision guidance models for microservice monitoring as a first step in the process of making such models available for broader discussion and validation.

References

[1] E. Wolff, Microservices: Flexible Software Architecture. Addison-Wesley Professional, 2016.

[2] C. Pautasso, O. Zimmermann, M. Amundsen, J. Lewis, and N. Josuttis, "Microservices in Practice, Part 1: Reality Check and Service Design," IEEE Softw., vol. 34, no. 1, pp. 91–98, Jan. 2017.

[3] S. Newman, Building Microservices. O'Reilly Media, Inc., 2015.

[4] J. Lewis and M. Fowler, "Microservices," 25-Mar-2014. [Online]. Available: https://martinfowler.com/articles/microservices.html.

[5] G. A. Lewis, P. Lago, and P. Avgeriou, "A Decision Model for Cyber-Foraging Systems," in 13th Working IEEE/IFIP Conference on Software Architecture (WICSA), 2016, pp. 51–60.

[6] O. Zimmermann, "Guidance Models and Decision-Making Tooling for SOA, Cloud, and Outsourcing Solution Design," 2011.

[7] R. Weinreich and W. Kurschl, "Dynamic analysis of distributed object-oriented applications," in Proceedings of the Thirty-First Hawaii International Conference on System Sciences, 1998, vol. 7, pp. 386–394 vol.7.

[8] P. Aichinger, B. Mayer, and R. Weinreich, "Microservice Dashboard." [Online]. Available: https://www.se.jku.at/microservice-dashboard/. [Accessed: 01-Mar-2017].

[9] "Digital Performance & Application Performance Monitoring," Dynatrace. [Online]. Available: https://www.dynatrace.com/. [Accessed: 02-Mar-2017].

[10] "New Relic: Application Performance Management and Monitoring," New Relic. [Online]. Available: https://newrelic.com/. [Accessed: 02-Mar-2017].

[11] "OpenZipkin - A distributed tracing system," Zipkin. [Online]. Available: http://zipkin.io. [Accessed: 02-Mar-2017].

[12] "Open Source Elastic Stack," Elastic Stack. [Online]. Available: https://www.elastic.co. [Accessed: 02-Mar-2017].

[13] F. Shull, J. Singer, and D. I. Sjøberg, Guide to advanced empirical software engineering, vol. 93. Springer, 2008.

[14] M. B. Miles and A. M. Huberman, Qualitative data analysis: An expanded sourcebook. Sage, 1994.

[15] B. Plattner and J. Nievergelt, "Monitoring program execution: A survey," Computer, vol. 14, no. 11, pp. 76–93, 1981.

[16] S. Benbernou et al., "State of the art report, gap analysis of knowledge on principles, techniques and methodologies for monitoring and adaptation of SBAs," -Cube Consort. Deliv. PO-JRA-12, vol. 1, 2008.

[17] M. Mansouri-Samani and M. Sloman, "Monitoring Distributed Systems," Netwrk Mag Glob. Internetwkg, vol. 7, no. 6, pp. 20–30, Nov. 1993.

[18] B. H. Sigelman et al., "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure," Google, Inc., 2010.

[19] M. K. Aguilera, J. C. Mogul, J. L. Wiener, P. Reynolds, and A. Muthitacharoen, "Performance Debugging for Distributed Systems of Black Boxes," in Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles, New York, NY, USA, 2003, pp. 74–89.

[20] A. MacLean, R. M. Young, V. M. E. Bellotti, and T. P. Moran, "Questions, Options, and Criteria: Elements of Design Space Analysis," Hum-Comput Interact, vol. 6, no. 3, pp. 201–250, 1991.

[21] A. MacLean and D. McKerlie, "Design Space Analysis and use-representations," in Scenario-Based Design: Envisioning Work and Technology in System Development, 1995.

[22] U. Zdun, "Systematic Pattern Selection Using Pattern Language Grammars and Design Space Analysis," Softw. Pract. Exp., vol. 37, no. 9, pp. 983–1016, Jul. 2007.

Authorized licensed use limited to: University of Applied Sciences Karlsruhe c/o KIT Library. Downloaded on October 28,2026 at 10:55:38 UTC from IEEE Xplore. Restrictions apply.